Day 9 — Advanced JavaScript

Complete answers to the Day 9 assignment. The task covers an interactive webpage, form/button events, dynamic data, searching/filtering, and asynchronous API responses.

1. index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Directory</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="container">
    <h1>Student Directory</h1>
    <form id="studentForm">
      <input id="studentName" placeholder="Student name" required>
      <input id="studentCourse" placeholder="Course" required>
      <button type="submit">Add Student</button>
    </form>
    <input id="searchInput" type="search" placeholder="Search students">
    <p id="message" aria-live="polite"></p>
    <ul id="studentList"></ul>
    <button id="loadUsers" type="button">Load Sample API Users</button>
    <p id="apiStatus" aria-live="polite"></p>
    <ul id="apiUsers"></ul>
  </main>
  <script src="script.js"></script>
</body>
</html>
```

2. style.css

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f3f5f8;
  color: #222;
}
.container {
  max-width: 700px;
  margin: 40px auto;
  padding: 24px;
  background: white;
  border-radius: 12px;
  box-shadow: 0 3px 14px #00000012;
}
h1 { text-align: center; }
form { display: flex; flex-wrap: wrap; gap: 8px; margin-bottom: 14px; }
input, button { padding: 10px; border: 1px solid #bbb; border-radius: 6px; }
form input { flex: 1; min-width: 150px; }
button { cursor: pointer; background: #2459a9; color: white; border: 0; }
#searchInput { width: 100%; margin: 8px 0; }
li { margin: 8px 0; padding: 8px; background: #f1f4f8; border-radius: 5px; }
#message, #apiStatus { min-height: 20px; }
```

3. script.js

```
let students = [
  { id: 1, name: "Aarav", course: "JavaScript" },
  { id: 2, name: "Mira", course: "HTML and CSS" },
  { id: 3, name: "Kabir", course: "JavaScript" }
];

const studentList = document.querySelector("#studentList");
const studentForm = document.querySelector("#studentForm");
const nameInput = document.querySelector("#studentName");
const courseInput = document.querySelector("#studentCourse");
const searchInput = document.querySelector("#searchInput");
const message = document.querySelector("#message");

const renderStudents = (items = students) => {
  studentList.replaceChildren();
  items.forEach(({ name, course }) => {
```

```
    const li = document.createElement("li");
    li.textContent = `${name} — ${course}`;
    studentList.appendChild(li);
  });
};

studentForm.addEventListener("submit", (event) => {
  event.preventDefault();
  const name = nameInput.value.trim();
  const course = courseInput.value.trim();

  if (!name || !course) {
    message.textContent = "Please enter both name and course.";
    return;
  }

  const newStudent = { id: Date.now(), name, course };
  students = [...students, newStudent];
  renderStudents();
  studentForm.reset();
  message.textContent = "Student added successfully!";
});

searchInput.addEventListener("input", () => {
  const term = searchInput.value.trim().toLowerCase();
  const filtered = students.filter(({ name, course }) =>
    name.toLowerCase().includes(term) ||
    course.toLowerCase().includes(term)
  );
  renderStudents(filtered);
  message.textContent = `${filtered.length} student(s) found.`;
});

console.log("Names:", students.map(({ name }) => name));

const counts = students.reduce((result, { course }) => {
  result[course] = (result[course] || 0) + 1;
  return result;
}, {});
console.log("Course counts:", counts);

const total = (...scores) => scores.reduce((sum, score) => sum + score, 0);
console.log("Total:", total(10, 20, 30));

const waitBriefly = () => new Promise(resolve =>
  setTimeout(() => resolve("Promise completed."), 500)
);
waitBriefly().then(result => console.log(result));

async function loadApiUsers() {
  const status = document.querySelector("#apiStatus");
  const list = document.querySelector("#apiUsers");
  status.textContent = "Loading users...";
  list.replaceChildren();

  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!response.ok) throw new Error(`HTTP error: ${response.status}`);
    const users = await response.json();

    users.slice(0, 5).forEach(({ name, email }) => {
      const li = document.createElement("li");
      li.textContent = `${name} — ${email}`;
      list.appendChild(li);
    });
    status.textContent = "Sample users loaded successfully.";
  } catch (error) {
    status.textContent = "Could not load users. Check your internet and try again.";
    console.error(error);
  }
}

document.querySelector("#loadUsers").addEventListener("click", loadApiUsers);
renderStudents();
```

4. How to run

1. Create a folder named Day9AdvancedJavaScript.
2. Save the three code sections as index.html, style.css, and script.js in that folder.
3. Open index.html in a browser. For reliable API testing, use VS Code Live Server or another local web server.

4. Add a student using the form; the list updates without reloading.
5. Search by student name or course.
6. Click “Load Sample API Users” to fetch up to five names and email addresses. Internet access is required.

5. Concepts used

DOM manipulation: selecting and changing HTML elements.
Event handling: responding to form submission, typing, and button clicks with addEventListener().
map(): transforms each array item.
filter(): returns items matching a condition.
reduce(): combines array items into a single result, such as a count or total.
Arrow functions: short function syntax using =>.
Destructuring: extracts properties such as name and course.
Spread operator: expands values, as in [...students, newStudent].
Rest operator: collects arguments, as in (...scores).
Promise: represents an operation that may complete later.
async/await: makes Promise-based asynchronous code easier to read.
Fetch API: requests data from an API.

6. Expected output

The page displays three initial students: Aarav — JavaScript, Mira — HTML and CSS, and Kabir — JavaScript. Adding a student updates the list. Searching filters by name or course. If the API request succeeds, up to five sample users and their email addresses appear. The browser console shows the names array, course counts, total 60, and “Promise completed.”